



Standard User & Operator Manual + Technical Reference Document

xMixing Control System — Mitr Phol Production Plant

Document No.	MAN-PRD-MIX-MASTER-001
Revision	01 — Master Edition
Effective Date	25 June 2026
Prepared By	Engineering, IT & Production Department
Language	English Edition

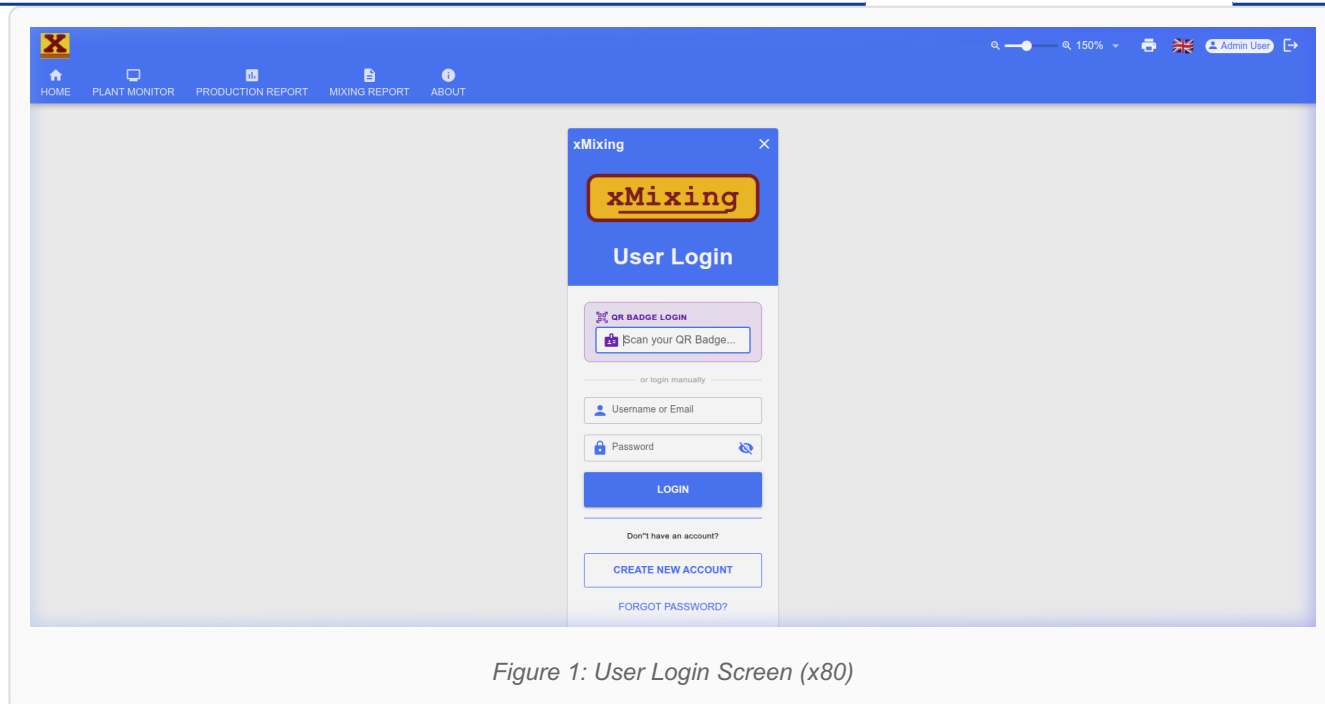
Table of Contents

Chapter	Topic
1	Operator Work Instruction (WI)
2	Barcode Scanning & Error-Proofing Workflow
3	Mixing Control Screen (x61) Usage Guide
4	Process Control Overview
5	Step Confirmation System
6	PLC Emergency Recovery Procedure
7	Hardware Datasheet — Siemens S7-1200 PLC
8	Hardware Datasheet — PLC Memory Map
9	PLC Recipe Architecture & SKU Analysis
10	System Architecture Overview
11	Production Deployment Guide
12	Complete Standard Workflow

Chapter 1: Illustrated Work Instruction

1.1 User Login (x80)

Purpose: Verify operator identity before system access. The system logs the operator name for every Batch.



How to use: 1. ใช้ป้อนสแกนบาร์โค้ดยังบัตรพนักงาน **หรือ** กรอก Username + Password ด้วยตนเอง 2. กดปุ่ม [เข้าสู่ระบบ / Login] สีน้ำเงิน 3. ชื่อพนักงานจะปรากฏที่มุมขวาบนของทุกหน้า

Button Functions:

ปุ่ม	หน้าที่
เข้าสู่ระบบ (Login)	ยืนยันตัวตนและเข้าสู่ระบบ
ออกจากระบบ (Logout)	บันทึกออกจากระบบ (กดที่ชื่อมุมขวาบน)

1.2 Production Plan (x55)

Purpose: View all assigned production batches and their current status.

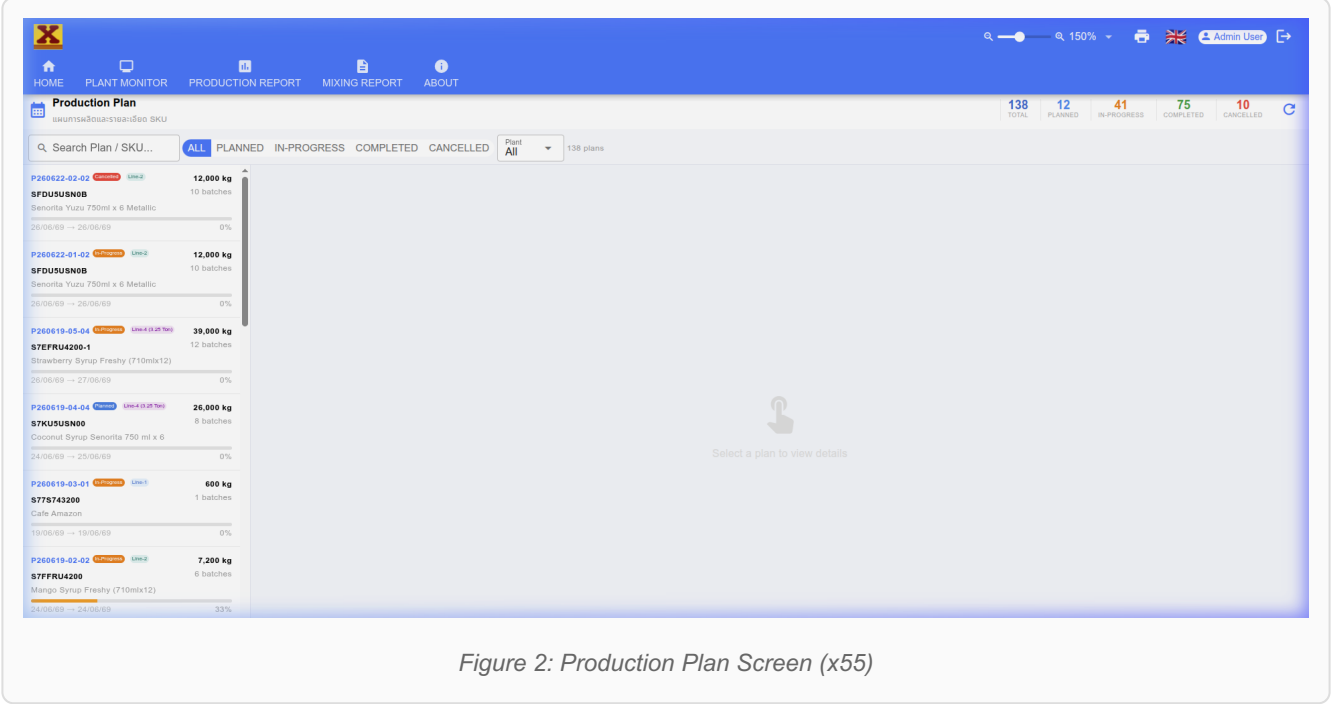


Figure 2: Production Plan Screen (x55)

How to use: 1. สังเกตคอลัมน์ **Status** — เลือก Batch ที่มีสถานะ **Prepared** (สีเขียว) 2. กดปุ่ม **[Select / เลือก]** ที่แถว 3. ระบบจะโหลดข้อมูล Batch ไปยังหน้า Mixing Control อัตโนมัติ

Button Functions:

ปุ่ม	หน้าที่
Select	เลือก Batch เพื่อเริ่มผลิต
+ New Plan	เพิ่มแผนการผลิตใหม่ (Supervisor)
ค้นหา	กรองรายการตามชื่อสินค้าหรือวันที่
สถานะ (Status Badge)	Created / Prepared / In-Progress / Done

1.3 SKU View (x56)

Purpose: Review Bill of Materials (BOM) and production steps (Phase/Step) before starting production.

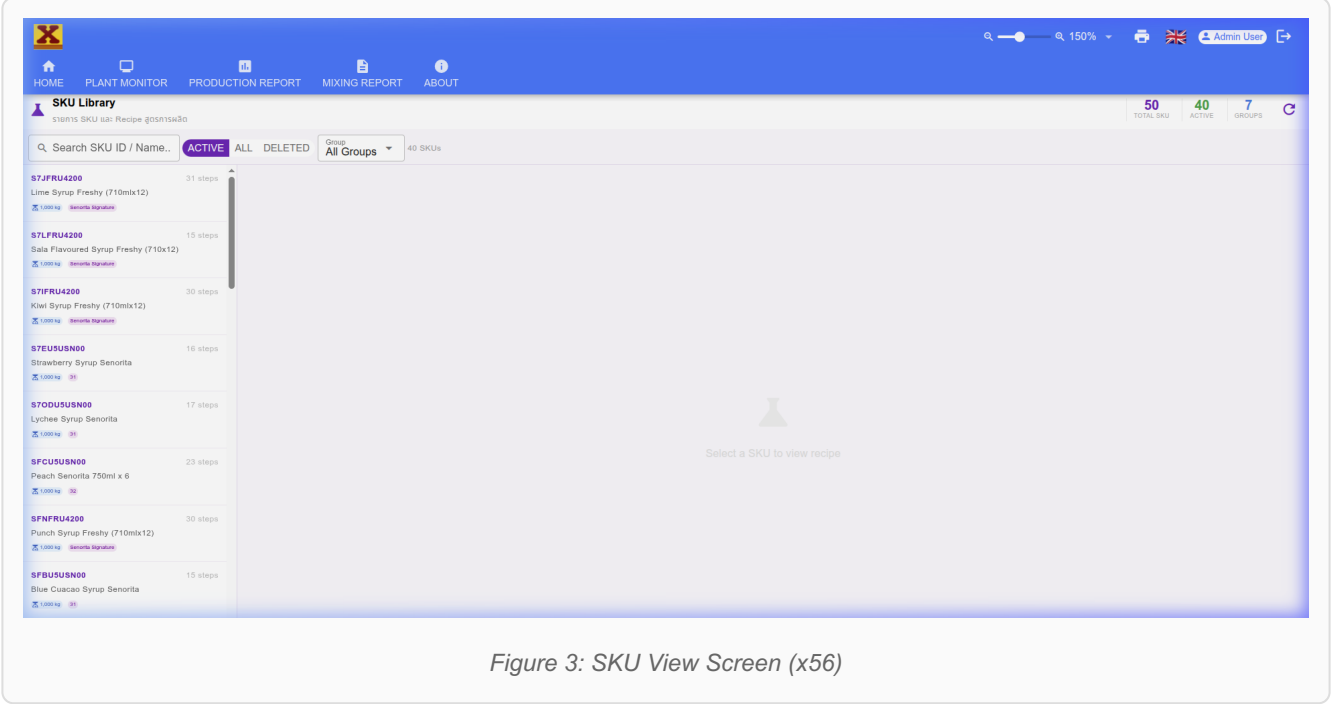


Figure 3: SKU View Screen (x56)

How to use: 1. ค้นหาสินค้าในช่องค้นหา 2. คลิกที่ชื่อสินค้าเพื่อดูรายละเอียด BOM 3. ตรวจสอบรายการวัตถุดิบ, น้ำหนัก, Phase, และ Step ให้ตรงกับใบสั่งผลิต

Button Functions:

ปุ่ม	หน้าที่
ค้นหา SKU	กรองรายการสินค้า
ดูรายละเอียด	เปิดดู BOM และขั้นตอนการผลิต
Print Label	พิมพ์ Label สำหรับถุงวัตถุดิบ

1.4 Check for Production (x60)

Purpose: Weigh and confirm each ingredient before sending to the production line.

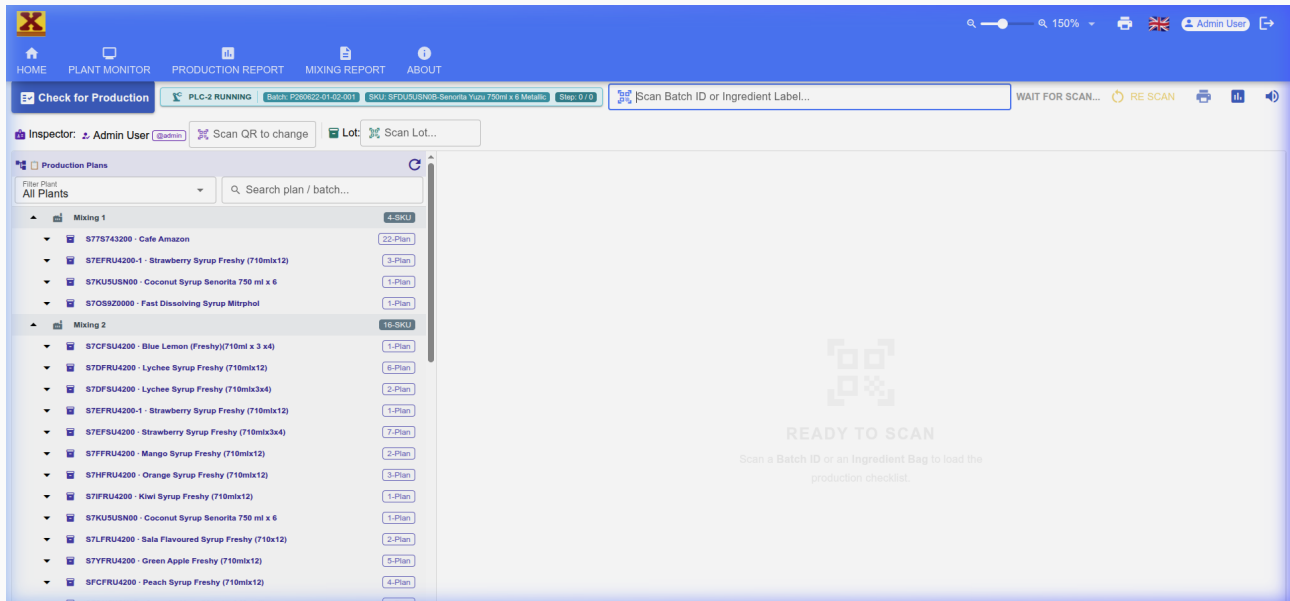


Figure 4: Check for Production Screen (x60)

How to use: 1. เลือก Batch ที่ต้องการเตรียม 2. ชั่งน้ำหนักวัตถุดิบแต่ละรายการให้ได้ตาม **Require WT (kg)** 3. กดปุ่ม [✓ Confirm] ท้ายแต่ละแถว เมื่อน้ำหนักอยู่ในเกณฑ์ Tolerance 4. เมื่อ Confirm ครบทุกรายการ สถานะ Batch จะเปลี่ยนเป็น Prepared

Button Functions:

ปุ่ม	หน้าที่
✓ Confirm	ยืนยันน้ำหนักวัตถุดิบรายการนั้น
au / Reset	ยกเลิกการ Confirm รายการนั้นใหม่
Go to Mixing	ข้ามไปหน้าควบคุมการผสมทันที
แถบสี WH	แดง = SPP, น้ำเงิน = FH, เขียว = MIX

1.5 Mixing Control (x61)

Purpose: Main control screen for operators to control the mixing machine and scan ingredients step by step.

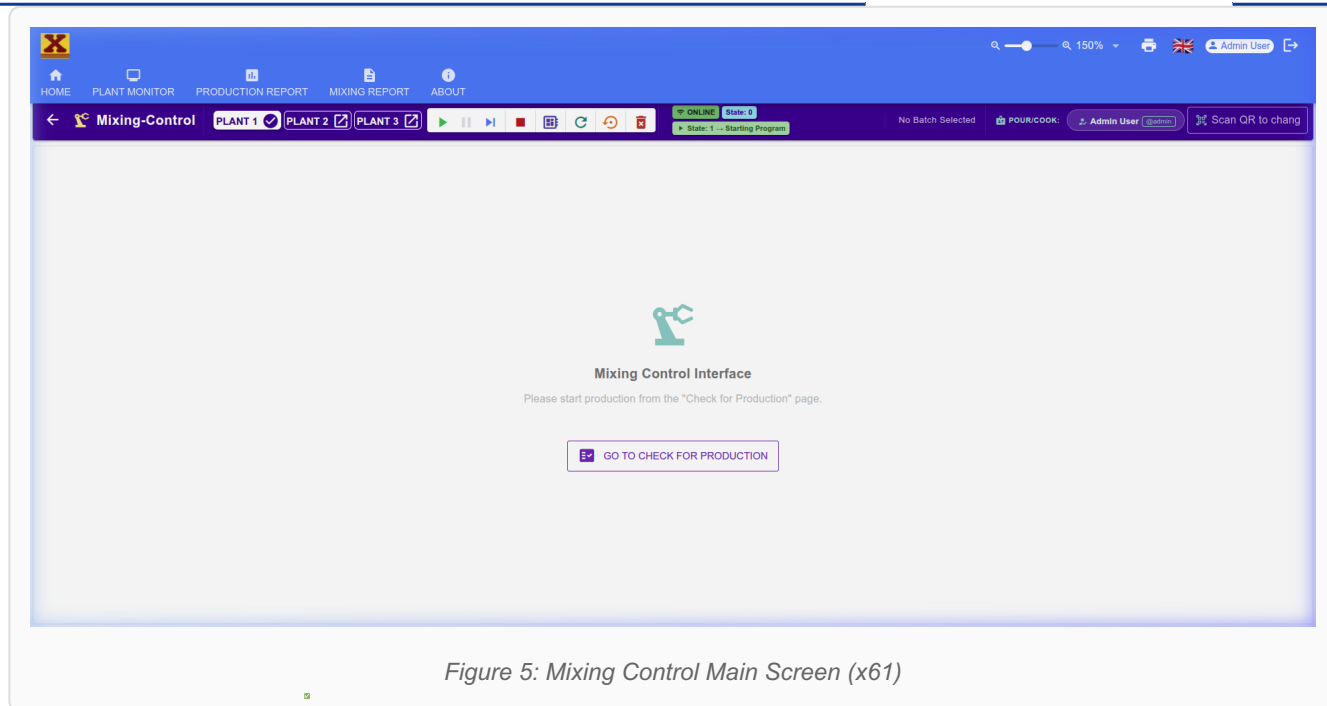


Figure 5: Mixing Control Main Screen (x61)

How to use: 1. เลือก Plant (1, 2 หรือ 3) ที่จะใช้งาน — ดูป้ายสถานะ **PLC CONNECTED** 2. เลือก Batch ที่มีสถานะ **Prepared** จากรายการ 3. กดปุ่ม **[START]** เพื่อเริ่มเดินเครื่อง 4. ระบบจะแสดงขั้นตอน (Step) ที่กำลังทำงาน — **แถบสีเขียว = กำลังดำเนินการ / แถบสีเหลือง = รอสแกน** 5. เมื่อขึ้นสัญญาณรอสแกน ให้ใช้ปืนสแกนยิง QR Code ที่ถูกวัดทุกติบ 6. ระบบยืนยันด้วยเครื่องหมาย และดำเนินการอัตโนมัติ

Control Button Functions:

ปุ่ม	ไอคอน	หน้าที่
START / RESUME	■	เริ่มหรือทำต่อหลัง Hold
PAUSE / HOLD	■	หยุดชั่วคราว PLC รักษาค่าเดิมไว้
STOP / ABORT	■	ยกเลิก Batch ทันที (ดูเงื่อนไขเท่านั้น)
BYPASS ()	■	ข้ามขั้นตอน — ต้องสิทธิ์ Supervisor
PLANT 1/2/3		สลับเครื่องจักรที่ควบคุม
Scan QR		เปิดกล้อง/ช่องสแกนบาร์โค้ด
Refresh		โหลดข้อมูลจาก Server ใหม่

Step Row Color Indicators:

สี	ความหมาย
เขียว	Step ที่กำลัง Active อยู่

สี	ความหมาย
เหลือง	รอ Operator สแกนวัตถุดิบ
เทา/เครื่องหมาย	Step ที่ทำสำเร็จแล้ว
แดง	Error / ค่าเกิน Tolerance

1.6 Plant Monitor (x100)

Purpose: Real-time overview of all Plant machine statuses for Supervisors.

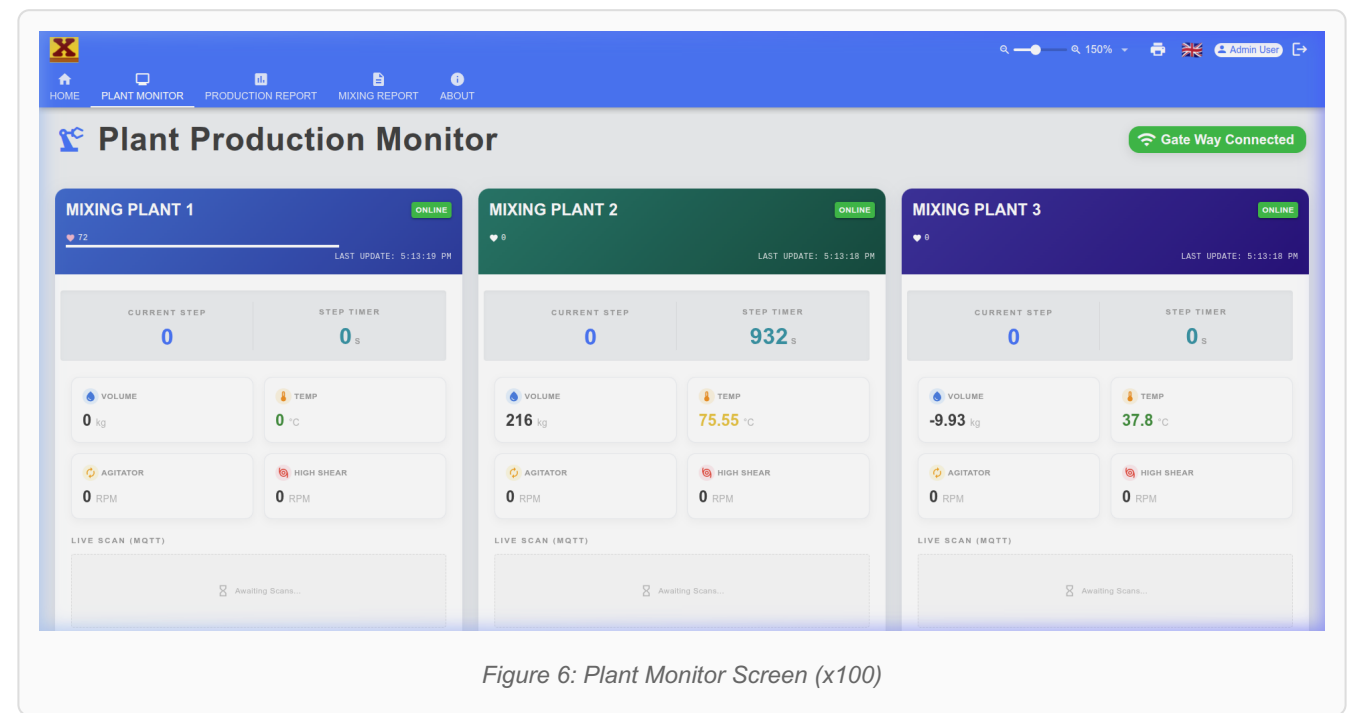


Figure 6: Plant Monitor Screen (x100)

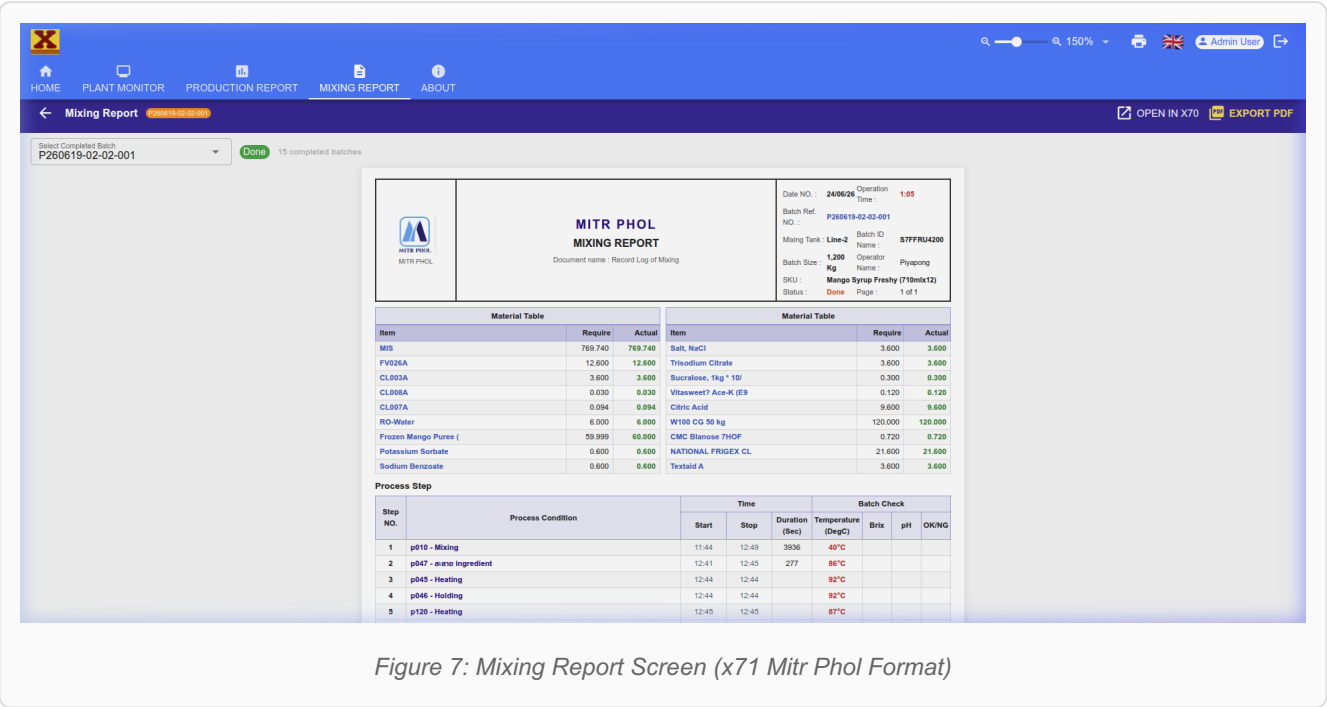
How to use: 1. เปิดหน้านี้บนหน้าจอ Supervisor Station เพื่อดูภาพรวม 2. สีการ์ดแต่ละ Plant บอกสถานะปัจจุบัน 3. คลิกที่การ์ดเพื่อดูรายละเอียดหรือกระโดดไปหน้า Mixing Control ของ Plant นั้น

Status Colors:

สี	สถานะ
เขียว	Running — กำลังผลิต
เหลือง	Hold — หยุดชั่วคราว
แดง	Alarm / Error
เทา	Standby — ว่างพร้อมรับงาน

1.7 Production & Mixing Report (x70, x71)

Purpose: View production summaries, actual ingredient quantities, and historical QC values.



How to use: 1. เลือกรายงานที่ต้องการ: **Production Report (x70)** หรือ **Mixing Report (x71)** 2. ค้นหา Batch ID หรือเลือกจากรายการ 3. กดปุ่ม **[Print / Export]** เพื่อพิมพ์หรือส่งออกไฟล์

Button Functions:

ปุ่ม	หน้าที่
Print	พิมพ์รายงานฉบับ Mitr Phol
Export PDF	บันทึกเป็นไฟล์ PDF
ค้นหา Batch	กรองรายงานตาม Batch ID หรือวันที่

Chapter 2: Barcode Scanning & Error-Proofing Workflow

Barcode Scanning & Verification Workflow

This diagram illustrates the step-by-step logic the application follows when an operator scans a QR code using the handheld scanner. It highlights the error-handling logic for various scanning mistakes, including the Wrong Box handling.



Chapter 3: Mixing Control Screen (x61) Usage Guide

x61 - Mixing Control Page Concept (Plant 1)

หน้านี้คือ **หัวใจหลักของการดำเนินงาน (Operation Control)** เป็นหน้าจอที่ Operator ใช้คุมเครื่องจักรและตรวจสอบสถานะทุกอย่างระหว่างการผลิต โดยในเฟสนี้เราจะโฟกัสที่ **Plant 1 (Mixing Tank 1)** เป็นหลัก

1. หัวใจหลักของหน้า x61 (Core Concepts)

- Safety & Clarity FIRST** : ต้องเห็นค่าสำคัญชัดเจน (อุณหภูมิ, น้ำหนัก) และมีปุ่มฉุกเฉิน (ABORT) ที่เรียกใช้งานได้ง่าย
- Step-by-Step Handshake** : หน้าเว็บจะเป็นคนจ่ายงาน (Recipe) ที่จะบรกดให้ PLC และรอจนกว่า PLC จะทำสเตปนั้นเสร็จ แล้วจึงไปต่อ
- Actual vs Setpoint Comparison** : โชว์เปรียบเทียบค่าที่ควรจะเป็น (SP) กับค่าที่วัดได้จริงจากเซนเซอร์ตลอดเวลา
- Mandatory QC Stop** : ถ้าเป็น Step ที่เกี่ยวกับการเย็นลง (Cooldown/Final) ระบบต้องหยุดให้ Operator กรอกค่า Brix / pH ไม่จับไม่ให้อัน Batch

2. โครงสร้างหน้าจอ (UI Layout Design)

หน้าจอออกแบบมาให้ Operator เข้าใจง่าย แบ่งเป็นโซนชัดเจน (3 โซนหลัก):

Zone A: Header & Status (ด้านบนสุด)

- วัตถุประสงค์: ข้อมูลแผนการผลิต
- ข้อมูลที่แสดง:
 - ชื่อ SKU และรหัสสินค้า (เช่น S77S743200 - Cafe Amazon)
 - หมายเลข Batch และ Target Weight (เช่น 1200kg)
 - เชื่อมต่อ PLC สำเร็จหรือไม่ (แสดงป้าย PLC CONNECTED)

Zone B: Live Telemetry Dashboard (ตรงกลางจอ)

- วัตถุประสงค์: แสดงสถานะเครื่องจักร Real-time ทุก 1 วินาที
- ข้อมูลที่แสดง (แบ่งเป็น 4 คอลัมน์):
 - Hopper Scale**: น้ำหนักของสารระเหย หรือสารปรุงแต่งที่รอโหลด
 - Mixing Tank (ถังหลัก)**: อุณหภูมิน้ำ, น้ำหนักรวม, ความเร็วใบกวน Agitator
 - High Shear**: ความเร็วใบมีดบดละเอียด และอุณหภูมิมอเตอร์
 - Circulation**: ความเร็ว Flow Pump
- จุดเด่น**: ในส่วนนี้จะมีย้ายตัวเลขเล็กๆ โชว์คำว่า SP: 40°C เป๊ะไว้ข้างๆ Actual: 42°C ถ้าอยู่ในเกณฑ์ $\pm 5^{\circ}\text{C}$ จะเป็นสีเขียว ถ้าเลยเกณฑ์จะเป็นสีแดง

Zone C: Operation & Command Center (ด้านล่าง)

- วัตถุประสงค์: การควบคุม และบอกพิกัดว่าถึงขั้นไหนแล้ว
- ข้อมูลที่แสดง:
 - **Command Buttons:**
 - `START` : จำงาน Step ปัจจุบันให้ PLC รับ
 - `II PAUSE` : สั่งให้ PLC ะงับการขยับ ทุกอย่างหยุดหมุน
 - `ABORT` : เบรกดูกเงิน เคลียร์ระบบ
 - `Force Next Step` : ข้ามสเตปแบบบังคับ (ใช้กรณีฉุกเฉิน/Manual Override)
 - **SKU Step List (Accordion):** แยกกล่องทีละ Phase (เช่น Phase A1010) แล้วแสดงว่า PLC ตอนนี้อยู่ที่ขั้นตอนไหน (Highlight) ให้รู้ว่า “กำลังกวนตัวนี้อยู่”

3. ลำดับการควบคุมจากหน้า x61 (Operation Flow)

1. **Initiate (โหลดข้อมูล):**
 - เมื่อเข้ามาจากหน้า x60 ระบบจะดึง “ทุก Step” ไล่ตารางเก็บไว้
 - เชื่อมต่อ MQTT รอรับค่าจาก Plant 1
2. **Execute (การรันแบบปกติ):**
 - Operator กด `START`
 - หน้า x61 ส่งข้อมูล Step ที่ 1 เท่านั้น ไปที่: `mixing/plant/1/step_cmd`
 - รอรับข้อความ `status: STEP_COMPLETE` จาก PLC (วนลูปแบบนี้จนจบ)
3. **The QC Trap (ขั้นตอนบังคับ):**
 - เมื่อ Step ใดไปจนถึงกลุ่ม Cooldown (`x1030` , `x1040`)
 - หน้าจอจะค้าง ไม่ Auto-advance
 - มีกล่อง Input โผล่มาให้กรอก: `Actual Brix [ ]` และ `Actual pH [ ]`
 - ต้องกรอกแล้วกด Save -> ระบบบันทึกลง DB -> ค่อยวิ่งต่อไป
4. **Conclusion (จบงาน):**
 - เมื่อถึงบรรทัดสุดท้าย โชว์ข้อความสำเร็จ พิมพ์เอกสารสรุป Batch Report อัตโนมัติ

4. ลอจิกการคุยกับระบบหลังบ้าน (Backend API / Database)

เนื่องจากต้องบันทึกประวัติ (History) หน้า x61 นี้จะเป็นจุดเดียวที่ยิง API ลงฐานข้อมูล:

- **เมื่อทำ Finished 1 Step:**
 - `POST /api/records/step`
 - Payload: `{ batch_id, step_id, actual_temp_end, duration_seconds }`
- **เมื่อกด Confirm QC Check:**
 - `POST /api/records/qc`
 - Payload: `{ batch_id, phase_id, actual_brix, actual_ph }`
- **เมื่อรันถึง Step สุดท้าย:**
 - `POST /api/batch/complete`

สรุป (Conclusion)

หน้า **x61-MixingControl** ถูกออกแบบให้เป็น “**HMIอัจฉริยะ**” ไม่ใช่แค่หน้าจออมิเตอร์ แต่ทำหน้าที่ป้อนคำสั่งให้ PLC (S7-1200) ทำงานทีละคำสั่ง (Micro-management) ข้อดีคือลดภาระ Memory ของ PLC และช่วยให้ฝัง Software เก็บ Snapshot ข้อมูลทุก Step ได้ละเอียดที่สุด

Chapter 4: Process Control Overview



Process Control Concept — xMixing Production

1. Process Flow Overview

จากการวิเคราะห์ SKU ทั้ง 16 ตัวจริงจากฐานข้อมูล พบรูปแบบการผลิตดังนี้:

```
graph TD
    subgraph "Phase A - Auto Batching"
        A1[A1010<br>LS/Water/Sugar<br>Batching 40°C]
        A2[A1020<br>CMC/Emulsifier<br>+ High Shear 60°C]
    end

    subgraph "Phase D - Dissolve & Mix"
        D1[D1010<br>ละลาย Ingredient<br>60-86°C]
        D3[D1030<br>เติมสารเคมี<br>Salt/Acid/Sweetener]
    end

    subgraph "Phase x - Transfer & Control"
        X1[x1010<br>Heating Up<br>86-92°C]
        X2[x1020<br>Holding Time<br>88°C x 300-900s]
        X3[x1030<br>Cooldown<br>75-78°C<br>Record Brix/pH]
        X4[x1040<br>Final Cooling<br>40-43°C<br>Record Brix/pH CTW]
    end

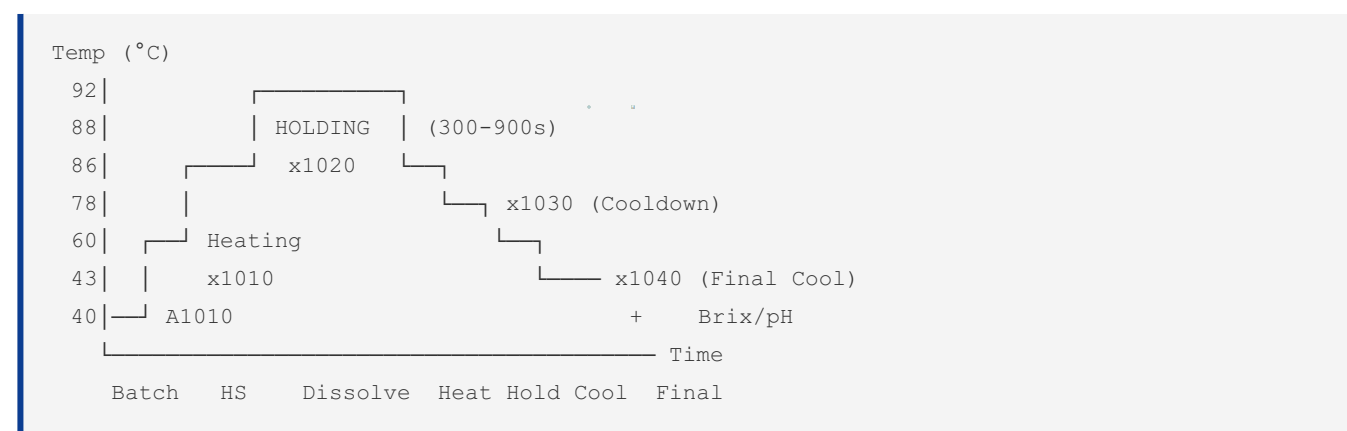
    A1 --> A2
    A2 --> D1
    D1 --> D3
    D3 --> X1
    X1 --> X2
    X2 --> X3
    X3 --> X4

    style A1 fill:#bbdefb
    style A2 fill:#bbdefb
    style D1 fill:#c8e6c9
    style D3 fill:#c8e6c9
    style X1 fill:#ffe0b2
    style X2 fill:#ffcdbc
    style X3 fill:#e1bee7
    style X4 fill:#b2ebf2
```

2. Temperature Zones (5 โซน)

Zone	Phase ID	อุณหภูมิ	Range	ระยะเวลา	วัตถุประสงค์
Mixing	A1010	40°C	-	ไม่กำหนด	ชั่งตวงวัตถุดิบหลัก (LS, Water, Sugar)
High Shear	A1020	60°C	-	120-180s	ผสม CMC, Emulsifier + High Shear 800-1700 RPM
Heating	x1010	86-92°C	85-95°C	ไม่กำหนด	ให้ความร้อนถึงจุดพาสเจอร์ไรส์
Holding	x1020	88°C	86-90°C	300-900s	คงอุณหภูมิฆ่าเชื้อ (Pasteurization)
Cooldown	x1030→x1040	78→43°C	40-80°C	ไม่กำหนด	ลดอุณหภูมิ + เติม Flavour + QC Check

Temperature Profile Chart:



3. Motor Control (2 ระบบ)

3.1 Agitator (ใบกวนหลัก)

สถานะ	RPM	ใช้ในช่วง
Low Speed	30 RPM	D1010, D1030 — ช่วงเติมวัตถุดิบละลาย
Normal Speed	60 RPM	A1020 — ช่วง High Shear ทำงาน
High Speed	80 RPM	A1010, x1010-x1040 — ช่วง Batch/Heat/Hold/Cool

3.2 High Shear (เครื่องบดละเอียด)

สถานะ	RPM	ใช้เมื่อ
OFF	0	ส่วนใหญ่ของกระบวนการ
Medium	800	เติมของเหนียว (Emulsifier, Textaid)
High	1700	เติม CMC, สารทำให้ข้น

[!IMPORTANT] High Shear ทำงานเฉพาะ Phase **A1020** เท่านั้น! และมี **timer control** (120-180 วินาที)

4. Recording & QC Checkpoints

4.1 ค่าที่ต้อง Record ตลอด (ทุก Step)

ค่า	Field	บันทึก	ประเภท
อุณหภูมิ	qc_temp	ทุก Step	Continuous
Steam Pressure	record_steam_pressure	ทุก Step	Continuous

4.2 ค่าที่ Record เฉพาะบาง Step (QC Checkpoint)

ค่า	Field	เมื่อไหร่	Phase ที่พบ
CTW (Cooling Tower Water)	record_ctw	ช่วง Final Cooling เท่านั้น	x1040
Brix	operation_brix_record	ช่วง Cooldown + Final	x1030, x1040
pH	operation_ph_record	ช่วง Cooldown + Final	x1030, x1040

4.3 QC Checkpoint Summary (จาก Blue Lemon Freshy 32 steps)

Step 1-23: ไม่มี Brix/pH record → เป็นขั้นตอนผลิตปกติ			
Step 24 (x1030 Cooldown 78°C):	Brix	pH	← QC Check #1
Step 25-31: ไม่มี			
Step 32 (x1040 Final 43°C):	Brix	pH	CTW ← QC Check #2 (Final)

5. Data Flow: Recipe → PLC → Record

```
sequenceDiagram
    participant DB as Central DB
    participant HMI as x31 Dashboard
    participant PLC as PLC Controller
    participant REC as Production Record DB

    Note over HMI: Operator clicks START
    HMI->>PLC: Download Recipe (ALL steps)

    loop Each Step
        PLC->>PLC: Execute step (dosing/heat/mix)
        PLC->>HMI: Telemetry (every 1s)<br>{temp, rpm, weight, timer}

        alt Step has record flag
            PLC->>HMI: Record Request<br>{actual_temp, actual_weight, actual_brix, actual_ph}
            HMI->>REC: Save Production Record
        end

        alt QC Checkpoint (x1030/x1040)
            PLC->>HMI: QC Data<br>{brix_actual, ph_actual, ctw}
            HMI->>HMI: Compare vs SP (Setpoint)
            HMI->>REC: Save QC Record
            Note over HMI: PASS or FAIL alert
        end
    end

    PLC->>HMI: Batch Complete
    HMI->>REC: Save Batch Summary
```

6. Production Record Schema

6.1 Per-Step Record (บันทึกทุก Step)

```
{
  "batch_id": "P260411-02-01-001",
  "step_no": 10,
  "phase": "p0010",
  "phase_id": "A1010",
  "re_code": "LS in Line",

  "setpoint": {
    "require": 371.71,
    "temperature": 40.0,
```

```
    "agitator_rpm": 80.0,
    "high_shear_rpm": 0
  },

  "actual": {
    "weight": 371.85,
    "temperature": 40.2,
    "agitator_rpm": 79.5,
    "high_shear_rpm": 0,
    "steam_pressure": 2.5,
    "elapsed_time": 45
  },

  "tolerance": {
    "weight_ok": true,
    "temp_ok": true
  },

  "timestamp_start": "2026-04-15T10:00:00",
  "timestamp_end": "2026-04-15T10:00:45",
  "recorded_by": "PLC"
}
```

6.2 QC Checkpoint Record (เฉพาะ x1030/x1040)

```
{
  "batch_id": "P260411-02-01-001",
  "checkpoint": "COOLDOWN",
  "phase_id": "x1030",

  "setpoint": {
    "temperature": 78.0,
    "temp_range": [75.0, 80.0],
    "brix_sp": "14.5",
    "ph_sp": "3.8"
  },

  "actual": {
    "temperature": 77.5,
    "brix": 14.3,
    "ph": 3.82,
    "ctw_temp": null,
    "steam_pressure": 2.1
  },

  "result": "PASS",
  "operator": "admin",
  "timestamp": "2026-04-15T10:15:30",
}
```

```
"notes": ""  
}
```

7. PLC Register Mapping Concept

Setpoint Registers (HMI → PLC) — Written per step

Register	Type	Description
DB100.0	INT	Step Number
DB100.2	INT	Action Code
DB100.4	REAL	Target Weight (kg)
DB100.8	REAL	Temp Setpoint (°C)
DB100.12	REAL	Temp Low Limit
DB100.16	REAL	Temp High Limit
DB100.20	REAL	Agitator RPM SP
DB100.24	REAL	High Shear RPM SP
DB100.28	INT	Step Time (s)
DB100.30	REAL	Low Tolerance
DB100.34	REAL	High Tolerance

Actual/Telemetry Registers (PLC → HMI) — Read every 1s

Register	Type	Description
DB200.0	INT	Current Step No
DB200.2	INT	Step Timer (s)
DB200.4	REAL	Mixing Tank Temp (°C)
DB200.8	REAL	Mixing Tank Weight (kg)
DB200.12	REAL	Agitator RPM Actual
DB200.16	REAL	High Shear RPM Actual
DB200.20	REAL	Hopper Weight (kg)

Register	Type	Description
DB200.24	REAL	Steam Pressure
DB200.28	REAL	CTW Temp
DB200.32	INT	Watchdog
DB200.34	INT	Status (0=IDLE, 1=RUN, 2=PAUSE, 3=DONE, 9=ERROR)

QC Registers (Operator Input or PLC Sensor)

Register	Type	Description
DB300.0	REAL	Actual Brix
DB300.4	REAL	Actual pH
DB300.8	REAL	Actual CTW Temp
DB300.12	BOOL	QC Record Trigger
DB300.13	BOOL	QC Pass/Fail

8. Implementation Plan

Phase 1: Recipe Download (Done)

- ☒ Send ALL recipe to PLC via MQTT
- ☒ PLC Simulator handles recipe
- ☒ Download button + ACK

Phase 2: Step-by-Step Telemetry (Current)

- ☐ PLC simulator uses recipe SP for realistic telemetry
- ☐ Dashboard highlights active step with actual vs SP
- ☐ Step auto-advance based on step_time

Phase 3: Production Records

- ☐ Create `production_records` table in DB

- ☐
API endpoint to save per-step records
- ☐
Auto-save actual values when step completes

Phase 4: QC Checkpoints

- ☐
Detect QC steps (brix/ph/ctw flags)
- ☐
Show QC input dialog at x1030/x1040
- ☐
Compare actual vs SP → PASS/FAIL
- ☐
Save QC records to DB

Phase 5: Batch Report

- ☐
Generate batch production report
 - ☐
Include all step records + QC results
 - ☐
Export as PDF for QA sign-off
-

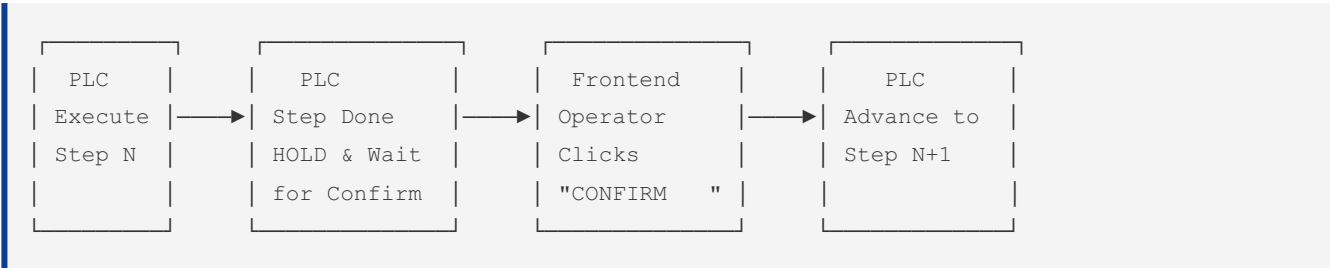
Chapter 5: Step Confirmation System

Step Confirmation Concept — Operator Must Confirm Every Step

The Problem

In the current “full auto” design, the PLC auto-advances after timer/condition is met. But in real food manufacturing, the **operator must visually verify** each step is actually done before moving forward (e.g., “Is the sugar fully dissolved?”, “Did the temperature reach 83°C?”).

The New Workflow: “Execute → Hold → Confirm → Advance”



PLC State Machine (Updated)

State 0: IDLE	→ Waiting for recipe download + Start command
State 1: EXECUTING	→ Running current step (timer counting, outputs ON)
State 2: WAIT_CONFIRM	→ Step finished, holding outputs, waiting for operator
State 3: PAUSED	→ Operator paused the batch
State 4: DONE	→ All steps complete
State 9: ERROR	→ Fault condition

Key Change: After every step completes, PLC goes to **State 2 (WAIT_CONFIRM)** instead of directly advancing. The agitator/temperature HOLD at current setpoint while waiting, so the product is safe.

Data Block Updates (DB1780)

Add these fields to the header:

```
// — Confirmation Handshake —
Step_Done      : Bool;    // PLC sets TRUE when step execution is finished
Confirm_Step   : Bool;    // PC sets TRUE to acknowledge and advance
```

```

Confirm_Phase_ID   : String[10]; // PC echoes back which phase it's confirming
Confirm_Step_ID    : Int;        // PC echoes back which step it's confirming

```

PLC Logic Change (in FB1780):

```

// After step timer/condition is met:
IF step_finished THEN
    "DB_RecipeData".Step_Done := TRUE;
    "DB_RecipeData".PLC_State := 2; // WAIT_CONFIRM
    // DO NOT advance – hold current outputs
    // Agitator keeps running, temperature maintains
END_IF;

// When PC confirms:
IF "DB_RecipeData".Confirm_Step AND "DB_RecipeData".PLC_State = 2 THEN
    // Verify the confirmation matches current step (safety!)
    IF "DB_RecipeData".Confirm_Phase_ID = current_phase
        AND "DB_RecipeData".Confirm_Step_ID = current_sub_step THEN

        // Reset flags
        "DB_RecipeData".Step_Done := FALSE;
        "DB_RecipeData".Confirm_Step := FALSE;

        // ADVANCE to next step
        advance_to_next_step();
        "DB_RecipeData".PLC_State := 1; // Back to EXECUTING
    END_IF;
END_IF;

```

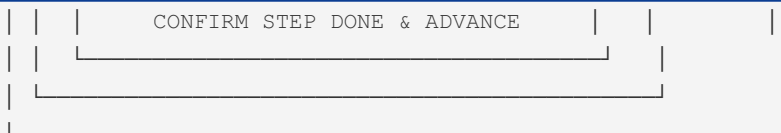
Frontend Changes (x62-MixingControlV2.vue)

What the Operator Sees:

```

| Step 5/13 – Phase p0030, Sub-Step 10 |
| Action: [20010] Dissolve Ingredient   |
| Material: Potassium Sorbate           |
|                                       |
| STEP EXECUTION COMPLETE               |
|                                       |
| Temperature: 60.2°C / 60.0°C          |
| Agitator:    1500 RPM                  |
| Duration:    5:00 / 5:00              |
|                                       |
|                                       |

```



Button Behavior:

1. **Button is DISABLED** while `PLC_State = 1` (Executing) — the step is still running
2. **Button turns GREEN** when `PLC_State = 2` (Wait_Confirm) — step is done, waiting for operator
3. **Operator clicks** → Frontend publishes MQTT:

```
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",
  "Confirm_Step_ID": 10,
  "timestamp": "2026-05-10T17:55:00"
}
```

4. PLC receives → validates → advances → `PLC_State` goes back to `1`
5. Frontend sees the state change and the next step starts highlighting

MQTT Message Flow

Step Execution (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 1,           ← Executing
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": false,
  "Step_Timer": 245        ← Countdown
}
```

Step Ready for Confirm (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 2,           ← Wait Confirm
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": true,        ← Step finished!
}
```

```
"Step_Timer": 0
}
```

Operator Confirms (Frontend → PLC):

```
Topic: mixing/plant/1/step_confirm
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",
  "Confirm_Step_ID": 10
}
```

PLC Advances (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 1,           ← Back to Executing
  "Phase_ID": "p0030",
  "Step_ID": 20,           ← Next step!
  "Step_Done": false,
  "Step_Timer": 300
}
```

Safety Rules

Rule	Description
Phase/Step Echo	Frontend MUST echo back which Phase+Step it's confirming. PLC rejects if mismatch.
Double-click Guard	Frontend disables the confirm button for 2 seconds after click to prevent double-advance.
Heartbeat Still Active	If PC_Heartbeat is lost while in WAIT_CONFIRM, PLC holds position indefinitely (safe state).
Timeout Warning	If WAIT_CONFIRM exceeds 10 minutes, PLC can trigger a warning alarm (but does NOT auto-advance).
Manual Steps	For ActionCode 21010 (Manual Add), the confirm button serves double duty: operator scans the ingredient AND confirms.

Summary of Changes Required

Layer	What to Change
PLC DB1780	Add <code>Step_Done</code> , <code>Confirm_Step</code> , <code>Confirm_Phase_ID</code> , <code>Confirm_Step_ID</code> fields
PLC FB1780	After step done → set State=2, hold outputs. On Confirm → validate → advance
Node-RED	Add S7-Out node to write <code>Confirm_Step</code> + IDs when MQTT confirm arrives
Frontend	Add “Confirm Step Done” button that enables only when <code>PLC_State = 2</code>
Mock Simulator	Update to wait for confirm MQTT message before advancing

Chapter 6: PLC Emergency Recovery Procedure

PLC & PC Concept and Recovery Workflow

Architecture for seamless recovery from App or PC shutdown during mixing execution.

To handle scenarios where the App or Computer shuts down abruptly while maintaining the ability to seamlessly recover, the best approach is the “**PLC as Executor, PC as Supervisor**” concept.

Once the production starts, the PLC must not rely on the PC to feed it instructions step-by-step. Instead, the PLC holds the recipe and manages the state machine, while the PC just watches and logs.

1. The Concept (PLC as State Master)

- **Pre-load the Recipe:** The entire recipe (or up to 50 steps) is downloaded into the PLC's `DB_Recipe` before the batch officially starts.
- **PLC Manages the Sequence:** The PLC uses an internal variable (e.g., `Current_Step_Idx`) to track where it is. It executes Step 1, finishes it, and automatically moves to Step 2 without asking the PC.
- **PC is just a Viewer/Logger:** The App continuously polls the PLC to read `Current_Step_Idx` and logs the actual values (actual temp, time taken, etc.) to the database.
- **Resume Capability:** Because the PLC knows the `Batch_ID` and the `Current_Step_Idx`, if the PC reboots, the PC simply asks the PLC: “*What Batch are you running, and what step are you on?*” and immediately syncs its UI back to reality.

2. Required Updates to your Data Block Header

To make recovery work, you need to add a few synchronization variables to your `DB_Recipe` header:

```
// Header Information
Batch_ID      : STRING[20];    // Critical: Links PLC memory to your Database Batch
SKU_ID       : STRING[20];
Total_Steps   : INT;
Current_Step_Idx : INT;        // PLC updates this as it moves forward

// Handshake & Status
Command_Start : BOOL;         // PC tells PLC to Start
Command_Pause : BOOL;         // PC tells PLC to Pause
PC_Heartbeat  : BOOL;         // Toggles every 1s. If PLC doesn't see toggle, PC is
dead.
PLC_State     : INT;           // 0=Idle, 1=Running, 2=Waiting for Manual, 3=Paused,
4=Done
```


3. Normal Workflow (No Crashes)

1. **Download:** Operator selects a Batch. The App writes the `Batch_ID` and the array of up to 50 steps to the PLC.
 2. **Start:** App sends `Command_Start = TRUE`.
 3. **Execution:** PLC reads `Steps[Current_Step_Idx]`, sets Agitator, High Shear, and Temp. It waits for the `Step_Time` to finish.
 4. **Transition:** When the time is up, PLC increments `Current_Step_Idx += 1` and executes the next step automatically.
 5. **Logging:** The App notices `Current_Step_Idx` changed, so it records the completion of the previous step into your `mixing_batch_step_log` table.
-

4. Recovery Workflow (App / Computer Crashes & Restarts)

Scenario: The PC dies during Step 15.

1. PLC Reaction (PC_Heartbeat fails):

- The PLC detects the `PC_Heartbeat` stopped toggling.
- **If the step is automatic** (e.g., mixing for 5 minutes), the PLC *continues* running the step to avoid ruining the product.
- **If the step requires manual scanning** (like Action 21010 “Manual Add”), the PLC safely goes into a `Waiting` state (keeping agitator on, but timer paused) because the operator needs the PC scanner to continue.

2. PC Reboots & App Starts:

- The App connects to the PLC and reads the `Batch_ID` and `Current_Step_Idx`.
- **App Logic:** “Ah, the PLC says it is running Batch `B20260510-01` at Step `15`.”

3. Syncing the Database:

- The App checks its database for `B20260510-01`.
- It sees the batch status is still `In Progress`.
- It sees logs up to Step 14. It knows Step 15 is currently active.

4. Resume UI:

- The App instantly routes the operator to the Mixing Control screen for that Batch, highlights Step 15 as “Running”, and resumes polling data. **Recovery Complete without any operator input.**
-

5. PLC Communication Protocol: The Best Choice

Since your architecture already utilizes **Node-RED** and **RabbitMQ**, the absolute most robust way to communicate with the PLC is to decouple the PC from the PLC physically using Node-RED as middleware.

Option A: S7 Protocol via Node-RED (Recommended for Siemens S7-1200/1500) * Use the `node-red-contrib-s7` node. * **Why:** It natively reads/writes to the Data Block (DB) based on absolute offsets (e.g., `DB1, X0.0`) without requiring any special code or licenses on the PLC side. It is extremely fast and lightweight.

Option B: OPC UA (Recommended if PLC supports it natively) * Use `node-red-contrib-opcua`. * **Why:** Tag-based and highly standardized. If the PLC tags move, OPC UA doesn't break.

The Ideal Communication Flow: 1. **Polling:** Node-RED connects to the PLC and polls the `DB_Recipe` header every 500ms. 2. **Publishing:** If `Current_Step_Idx` or `PLC_State` changes, Node-RED publishes a JSON message to a **RabbitMQ** topic (e.g., `plc.mixing.state`). 3. **Consuming:** FastAPI listens to RabbitMQ, logs the step completion securely into MySQL (`mixing_batch_step_log`), and broadcasts the live state to Nuxt.

6. Required Changes to Existing Applications

Backend (FastAPI) Updates

1. **Remove State-Machine Logic:** Remove any Python logic that says “Wait 5 minutes then trigger the next step”. FastAPI must become stateless; it only reacts to events emitted by the PLC via RabbitMQ.
2. **Add Download Endpoint:** Create a `POST /api/batch/{id}/download-to-plc` endpoint. This query formats the 50-step array and sends it to Node-RED to write to the PLC `DB_Recipe`.
3. **RabbitMQ Consumer Worker:** Create a background worker in FastAPI to consume PLC state changes from RabbitMQ and update the MySQL database instantly.

Frontend (Nuxt) Updates

1. **Make UI Stateless:** The UI must stop driving the process. Remove the manual “Next Step” buttons for any automated mixing/heating steps.
 2. **WebSocket / SSE Integration:** The frontend should connect to FastAPI via WebSocket. When the PLC moves to Step 15, the frontend instantly receives the event and visually highlights Step 15 in the UI.
 3. **Recovery on Mount (`onMounted`):** When the operator navigates to the Mixing Control page, the code should immediately fetch the current `PLC_State` and `Current_Step_Idx`. If the batch is already running, the UI simply jumps to that step and resumes monitoring.
 4. **Heartbeat Signal:** The Nuxt frontend (or FastAPI) must implement a loop that toggles the `PC_Heartbeat` bit in the PLC every 1-2 seconds so the PLC knows the PC is alive.
-

Chapter 7: Hardware Datasheet — Siemens S7-1200 PLC

S7-1200 PLC Data Block & Interaction Design

Method: Step-by-Step Handshake (via Node-RED & MQTT)

This document defines the data structures (Data Blocks) used for real-time communication between the HMI/ App and the Siemens S7-1200 PLC.

DB100: Step Command (App → PLC)

Role: Receives recipe parameters and execution commands for the current step. **Access:** App writes (via MQTT/Bridge), PLC reads.

Name	Data Type	Description	Example
Watch_Doc	Int	Watchdog timer/counter from App	1234
Plan_ID	String[20]	Production Plan Identifier	'PLAN-2024-001'
Batch_ID	String[20]	Current Production Batch	'P260411-02-01'
SKU_Name	String[30]	Product name or SKU identifier	'Strawberry_Jam_01'
Phase_ID	String[10]	Phase identifier code	'A1010'
Step_ID	Int	Unique Step ID	501
Step_Time_SP	Int	Target Dwell/Mixing time (seconds)	600
Step_Status	Int	0=Pending, 1=Active, 2=Complete	1
Material_ID	String[20]	Specific Raw Material ID	'MAT-099'
Re_Code_ID	String[20]	Ingredient or Recipe Code	'RM-SUG-01'
Req_Qty	Real	Required Quantity for step	250.5
TT_SP	Array[0..16] of Real	Temperature Profile Setpoints	[60.0, 65.0, ...]
Agitator_Speed	Real	Target Agitator Speed (RPM)	30.0
High_Shear_SP	Real	Target High Shear Speed (RPM)	1500.0
PH_Target	Real	Target pH level	4.5
Brix_Target	Real	Target Brix value	12.5
	Int		1

Name	Data Type	Description	Example
HMI_Command		0=IDLE, 1=START, 2=PAUSE, 3=ABORT	
Cmd_NewStep	Bool	Trigger Flag to start the step	TRUE

DB200: Telemetry (PLC → App)

Role: Live monitoring and feedback from the physical hardware. **Access:** PLC writes, App reads (polled or streamed every 1s).

Name	Data Type	Description	Example
Watchdog	Int	Heartbeat signal from PLC	32767
PLC_State	Int	0=Ready, 1=Run, 2=Hold, 9=Error	1
Current_Step	Int	Which step the PLC is actively executing	5
Step_Timer	Int	Elapsed time in current step (seconds)	124
Step_Status_Act	Int	Current Step Phase (e.g., 0=Init, 1=Dosing, 2=Mixing)	2
MixTank_Temp	Real	Live Tank Temperature (°C)	59.5
MixTank_Weight	Real	Live Tank Weight (kg)	249.8
Agitator_Act	Real	Live Agitator Speed (RPM)	30.1
HighShear_Act	Real	Live High Shear Speed (RPM)	1498.2
PH_Actual	Real	Live pH Sensor Reading	4.52
Brix_Actual	Real	Live Brix Sensor Reading	12.4
Hopper_Weight	Real	Live Sub-Hopper Weight (kg)	0.0

Interaction Flow (Handshake)

The communication follows a strict handshake protocol to ensure safety in manual and automatic modes.

```
sequenceDiagram
    participant App as Web Application
    participant NR as Node-RED / Bridge
    participant DB100 as PLC DB100 (CMD)
    participant DB200 as PLC DB200 (STATUS)

    Note over App,DB200: Step Execution Flow

    App->>NR: topic: plc/command (Step Data)
    NR->>DB100: Write All Params + Cmd_NewStep = 1

    Note over DB100: PLC Detects NewStep Trigger
    DB100->>DB100: Logic Starts Step
    DB100->>DB100: PLC Reset Cmd_NewStep = 0 (ACK)

    loop Every 1s
        DB200-->>NR: Current_Step, Temp, Timer
        NR-->>App: UI Update (Live Dashboard)
    end

    Note over DB200: PLC Finished Step
    DB200->>App: Step_Status_Act = Complete
```

Implementation Guidance

1. **Watchdog Logic:** Both the App (DB100) and PLC (DB200) increment their watchdog values. If a side detects that the other side's value hasn't changed for >5 seconds, it should trigger a **Communication Error** and safe-stop any active motor/pump.
2. **String Handling:** S7 strings use the first two bytes for length metadata. The App serialization must account for this (2 bytes + actual string data).
3. **Trigger Reset:** The PLC should reset `Cmd_NewStep` to `FALSE` as soon as it acknowledges the new parameters, serving as a handshake confirmation back to the App.

Chapter 8: Hardware Datasheet — PLC Memory Map

DB1511 Memory Map (S7-1200 Non-Optimized)

This document maps the exact byte offsets for **DB1511 (DB_FULL_RECIPE)**. This is extremely important for writing your `python-snap7 serialize()` function, as you must pack the binary payload to match these exact byte locations.

Important: This assumes the DB is set to `S7_Optimized_Access := 'FALSE'` (Standard Access) so offsets are deterministic.

1. `type_FullRecipe` (Header & Array)

This is the top-level structure of DB1511. The header takes up the first 52 bytes, followed by the array of 128 steps.

Offset	Name	Data Type	Size (Bytes)	Description
+0.0	Batch_ID	String[20]	22	Batch Code (e.g. "P260411-02"). S7 strings have 2 header bytes.
+22.0	SKU_ID	String[20]	22	Recipe/SKU Code
+44.0	HMI_Command	Int	2	0=IDLE, 1=START, 2=PAUSE, 3=ABORT, 9=RESET
+46.0	Total_Steps	Int	2	How many steps are actually populated (1-128)
+48.0	Active_Step	Int	2	PLC Pointer — current Seq number (1-128)
+50.0	Cmd_LoadRecipe	Bool	1 (bit 0)	Trigger bit to tell PLC a new array is ready
padding	(Padding)	(Byte)	1	S7 aligns Arrays to even byte boundaries

Offset	Name	Data Type	Size (Bytes)	Description
+52.0	Steps[1]	type_RecipeStep	78	First step (See Table 2)
+130.0	Steps[2]	type_RecipeStep	78	Second step
+208.0	Steps[3]	type_RecipeStep	78	Third step
...	...	...	78	... repeats ...
+9958.0	Steps[128]	type_RecipeStep	78	Last step

(Total Size of DB1511: **10,036 bytes** ≈ 10 KB)

2. type_RecipeStep (Individual Step Structure)

Each element in the Steps[1..128] array is exactly **78 bytes** long.

Relative Offset	Name	Data Type	Size (Bytes)	Description
+0.0	Seq	Int	2	Flat sequence number (1, 2, 3, ... 128)
+2.0	Phase_No	Int	2	Phase number (10, 20, 30...)
+4.0	Sub_Step	Int	2	Step within phase (10, 20, 30...)
+6.0	Action_Code	String[10]	12	Action code (e.g. "x10010")
+18.0	Phase_ID	String[10]	12	Phase ID (e.g. "p0010")
+30.0	Re_Code	String[20]	22	Ingredient / Material code
+52.0	Target_Weight	Real	4	Target dosing weight (kg)
+56.0	Temp_SP	Real	4	Temperature setpoint (°C)
+60.0	Temp_Low	Real	4	Min temperature limit (°C)

Relative Offset	Name	Data Type	Size (Bytes)	Description
+64.0	Temp_High	Real	4	Max temperature limit (°C)
+68.0	Agitator_SP	Real	4	Agitator speed (RPM)
+72.0	HighShear_SP	Real	4	High shear speed (RPM)
+76.0	Step_Time	Int	2	Hold time (Seconds)

(Total Size per Step: **78 bytes**)

3. Example: Cafe Amazon (S77S743200) — Flattened into Array

This is how the database recipe for SKU “Cafe Amazon” would be flattened into the `Steps[1..128]` array:

Seq	Phase_No	Sub_Step	Action_Code	Phase_ID	Ingredient	Target (kg)
1	10	10	x10010	p0010	LS in Line	371.71
2	10	20	x10020	p0010	RO-Water	372.87
3	10	30	x10030	p0010	White Sugar W150	250.0
4	20	10	x20010	p0020	(empty)	0.0
5	30	10	x30010	p0030	Potassium Sorbate	0.8
6	30	20	x30020	p0030	Sodium Benzoate	0.2
7	30	30	x30030	p0030	Malic Acid	0.115
8	30	40	x30040	p0030	Caramel Colour III	0.2
9	30	50	x30050	p0030	RO-Water	4.0
10	40	10	x40010	p0040	(empty)	0.0
11	45	10	x45010	p0045	Sugar Flavour SG-01	0.11
12	50	10	x50010	p0050	(empty)	0.0

Seq	Phase_No	Sub_Step	Action_Code	Phase_ID	Ingredient	Target (kg)
13	60	10	x60010	p0060	(empty)	0.0
14-128	0	0			(unused — Seq=0)	0.0

The PLC reads `Total_Steps = 13` , so it knows to stop after Seq 13 and ignore Steps[14] to Steps[128].

Python Serialization Tip

When you write the `.serialize()` function in `plc_interface.py`:

```
import struct

def pack_s7_string(s: str, max_len: int) -> bytes:
    s_bytes = s.encode('ascii')[:max_len]
    return struct.pack('BB', max_len, len(s_bytes)) + s_bytes.ljust(max_len, b'\x00')

# Pack Header (52 bytes)
header = b""
header += pack_s7_string(batch_id, 20)      # +0
header += pack_s7_string(sku_id, 20)       # +22
header += struct.pack('>h', hmi_command)   # +44
header += struct.pack('>h', total_steps)    # +46
header += struct.pack('>h', active_step)    # +48
header += struct.pack('?', cmd_load_recipe) # +50
header += b'\x00'                          # +51 padding

# Pack Each Step (78 bytes each)
for step in steps:
    header += struct.pack('>hhh', step.seq, step.phase_no, step.sub_step) # +0,2,4
    header += pack_s7_string(step.action_code, 10) # +6
    header += pack_s7_string(step.phase_id, 10)    # +18
    header += pack_s7_string(step.re_code, 20)     # +30
    header += struct.pack('>ffffff',
        step.target_weight, step.temp_sp, step.temp_low,
        step.temp_high, step.agitator_sp, step.highshear_sp) # +52..+76
    header += struct.pack('>h', step.step_time)    # +76

# Total payload = 52 + (128 * 78) = 10,036 bytes
plc.db_write(db_number=1511, start=0, data=header)
```

Chapter 9: PLC Recipe Architecture & SKU Analysis



SKU Recipe Analysis — All 16 Active SKUs

1. Full SKU Breakdown

SKU ID	Product Name	Steps	Phases	Max Steps/Phase	Avg Steps/Phase
S77S743200	Cafe Amazon	13	7	5	1.9
SFMU5USN00	Fresh Mint Senorita 750ml	15	8	4	1.9
S7GCA4SNA0	Japanese Melon Senorita 1900ml	17	8	8	2.1
S7GU5USN00	Japanese Melon Senorita 750ml	17	8	8	2.1
S7KCY4SNA0	Coconut Senorita 1900ml	17	9	4	1.9
S7KU5USN00	Coconut Senorita 750ml	17	9	4	1.9
S77CY4SN00	Classic Caramel Senorita	18	9	6	2.0
SFLDBUSS00	Rose Senorita Signature 720ml	24	10	7	2.4
S7EFRU4200-1	Strawberry Freshy 710ml x12	30	15	5	2.0
S7EFSU4200	Strawberry Freshy 710ml x3	30	15	5	2.0
S7HFRU4200	Orange Freshy 710ml x12	31	12	15	2.6
S7CFSU4200	Blue Lemon Freshy 710ml x3	32	17	5	1.9
S7DFRU4200	Lychee Freshy 710ml x12	34	16	5	2.1
S7DFSU4200	Lychee Freshy 710ml x3	34	16	5	2.1

SKU ID	Product Name	Steps	Phases	Max Steps/Phase	Avg Steps/Phase
S7YFRU4200	Green Apple Freshy 710ml x12	34	18	5	1.9
SFGFSU4200	Blue Hawaii Freshy 710ml x3	35	17	5	2.1

2. Statistical Summary

16 SKUs Analyzed			
Metric	Min	Max	Avg
Steps per SKU	13	35	24.9
Phases per SKU	7	18	12.1
Max steps in 1 phase	-	15	-
Avg steps per phase	1.9	2.6	2.1

3. Phase Type Classification

พบ 3 ประเภท Phase ในทุก SKU:

Prefix	Type	หน้าที่	ตัวอย่าง
A	Auto Batching	ซึ่งตรวจวัตถุดิบอัตโนมัติ (น้ำ, น้ำตาล, LS)	A1010, A1020
D	Dissolve / Mix	ละลาย, ผสม, ต้ม, เติมน้ำวัตถุดิบมือ	D1010, D1030
x	Transfer / Control	โอนย้ายถัง, รอ, QC, จบกระบวนการ	x1010, x1020, x1030, x1040

Process Flow Pattern (พบในทุก SKU):

<pre>graph LR A[A - Auto Batching
ซึ่งตรวจวัตถุดิบอัตโนมัติ] --> X1[x - Transfer
โอนเข้าถังผสม] X1 --> D[D - Dissolve
ละลาย/ผสม] D --> X2[x - Transfer
โอนออก] X2 --> X3[x1020 - QC Check]</pre>
--

X3 --> X4[x1030 - Storage]
X4 --> X5[x1040 - Complete]

4. Payload Size Analysis

Method	Worst Case	Typical	Comment
Send ALL	35 steps × 200B = 6.8 KB	25 × 200B = 4.9 KB	เล็กมาก!
Send per Phase	15 steps × 200B = 2.9 KB	2 × 200B = 0.4 KB	เล็กเกินไป
Send per Step	1 × 200B = 0.2 KB	-	ไม่คุ้ม overhead

[!IMPORTANT] ข้อมูลสำคัญ: Payload ของสูตรที่ใหญ่ที่สุด (35 steps) มีขนาดเพียง **6.8 KB** เท่านั้น — เล็กมากๆ สำหรับ PLC สมัยใหม่ที่มี memory เป็น MB ขึ้นไป

5. คำแนะนำ: Send ALL (Option A)

เหตุผล:

#	เหตุผล	รายละเอียด
1	Payload เล็กมาก	แม้ SKU ที่ซับซ้อนสุด (35 steps) ก็แค่ 6.8 KB — PLC รับได้สบายๆ
2	Avg 2.1 steps/phase	ถ้าส่งทีละ Phase จะต้อง handshake 12-18 ครั้ง เปลืองเกินไป
3	PLC ทำงานอิสระ	ไม่ต้องรอ Network ระหว่างขั้นตอน — ปลอดภัยกว่า สำหรับโรงงาน
4	ง่ายต่อการ Debug	ส่งครั้งเดียว ตรวจสอบว่าครบถ้วน แล้ว PLC วิ่งต่อเอง
5	Network Failure Safe	ถ้าเน็ต MQTT หลุดระหว่างผลิต PLC ยังวิ่งต่อได้

Architecture ที่แนะนำ:

```
sequenceDiagram
    participant OP as Operator
    participant HMI as x31 Dashboard
    participant MQTT as MQTT Broker
    participant PLC as PLC
```

```

OP->>HMI: Click "Start Production"
HMI->>HMI: Fetch all SKU steps from DB

HMI->>MQTT: mixing/plant/1/recipe<br>{batch_id, sku_id, batch_size,<br>steps: [all 13-35
steps]}

MQTT->>PLC: Full recipe (max 6.8 KB)
PLC->>MQTT: mixing/plant/1/ack<br>{status: "RECIPE_LOADED", steps: 35}

Note over HMI:      Recipe confirmed, PLC ready
OP->>HMI: Click "START"
HMI->>MQTT: mixing/plant/1/cmd<br>{command: "START"}

loop PLC executes autonomously
    PLC->>MQTT: mixing/plant/1/telemetry<br>{step_no, temp, rpm, weight, watchdog}
    HMI->>HMI: Update gauges in real-time
end

PLC->>MQTT: mixing/plant/1/status<br>{status: "BATCH_COMPLETE"}

```

MQTT Topic Design (Final):

```

mixing/plant/{id}/
├─ recipe          # HMI → PLC: Full recipe download (ALL steps)
├─ cmd             # HMI → PLC: START, PAUSE, RESUME, ABORT
├─ ack            # PLC → HMI: Recipe loaded / Command acknowledged
├─ telemetry       # PLC → HMI: Real-time sensor data (every 1-2s)
├─ status          # PLC → HMI: RUNNING, PAUSED, COMPLETE, ERROR
└─ watchdog        # PLC → HMI: Heartbeat counter

```

Recipe JSON Schema:

```

{
  "batch_id": "P260411-02-01-001",
  "sku_id": "S77S743200",
  "sku_name": "Cafe Amazon",
  "batch_size": 1200.0,
  "plant_id": 1,
  "total_steps": 13,
  "total_phases": 7,
  "timestamp": "2026-04-15T16:40:00",
  "steps": [
    {
      "step_no": 10,
      "phase": "p0010",
      "phase_id": "A1010",
      "action_code": "10030",
      "re_code": "LS in Line",
      "destination": "1010",

```



```
    "require": 371.71,  
    "uom": "kg",  
    "low_tol": 0.001,  
    "high_tol": 0.001,  
    "agitator_rpm": 80.0,  
    "high_shear_rpm": 0.0,  
    "temperature": 40.0,  
    "temp_low": 0.0,  
    "temp_high": 0.0,  
    "step_time": 0,  
    "step_timer_control": 0  
  }  
]  
}
```

6. Next Steps

- ☐ Implement recipe publish in `x61-MixingControl.vue` (Send All on Start)
- ☐ Update `plc_simulator.mjs` to receive & parse full recipe
- ☐ Add recipe ACK handling in `useMQTT.ts`
- ☐ Add step progress tracking on dashboard (current step highlight)